{ </> }

CODE MASTER

30 symboles + toutes les bases du code

. MAITRISE LES BASES. CONSTRUIS LE FUTUR .

( ) { } [ ] ; == => && %

- **SYMBOLES & OPERATEURS**
  pages 01 a 03
- **LOGIQUE : conditions, boucles, fonctions**
  pages 04 a 08
- **PYTHON & JAVASCRIPT en 1 page**
  pages 09 et 10
- **WEB : HTML + CSS**
  pages 11 et 12
- **OUTILS : Git, terminal, SQL, regex, HTTP**
  pages 13 a 17
- **RACCOURCIS + QUIZ + pense-bete**
  pages 18 a 20

AREEJ CODE

ton compagnon de revision - imprime, affiche, code

{ </> }

30 SYMBOLES DE PROGRAMMATION

que tout developpeur doit connaitre

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01
( )
PARENTHESES
Groupent et ordonnent les expressions.

02
{ }
ACCOLADES
Blocs de code ou objets.

03
[ ]
CROCHETS
Tableaux, listes, index.

04
;
POINT-VIRGULE
Termine une instruction.

05
:
DEUX-POINTS
Conditions, boucles, labels.

06
,
VIRGULE
Separe valeurs et arguments.

07
.
POINT
Accede aux proprietes.

08
=
AFFECTATION
Donne une valeur a une variable.

09
+
ADDITION
Additionne deux valeurs.

10
-
SOUSTRACTION
Soustrait une valeur.

11
*
MULTIPLICATION
Multiplie deux valeurs.

12
/
DIVISION
Divise une valeur.

13
%
MODULO
Reste d'une division.

14
==
EGAL A
Teste si deux valeurs sont egales.

15
!=
DIFFERENT DE
Teste si deux valeurs different.

16
>
SUPERIEUR A
Vrai si la gauche est plus grande.

17
<
INFERIEUR A
Vrai si la gauche est plus petite.

18
>=
SUPERIEUR OU EGAL
Plus grand ou egal.

19
<=
INFERIEUR OU EGAL
Plus petit ou egal.

20
&&
ET LOGIQUE
Vrai si les 2 conditions sont vraies.

21
||
OU LOGIQUE
Vrai si au moins une est vraie.

22
!
NON LOGIQUE
Inverse le booleen.

23
->
FLECHE
Fonctions, lambdas, etc.

24
=>
DOUBLE FLECHE
Fonctions flechees JS.

25
&
ET BIT A BIT
Operation AND binaire.

26
|
OU BIT A BIT
Operation OR binaire.

27
^
XOR BIT A BIT
OU exclusif binaire.

28
~
NON BIT A BIT
Inverse tous les bits.

29
<<
DECALAGE GAUCHE
Decale les bits a gauche.

30
>>
DECALAGE DROITE
Decale les bits a droite.

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

LES TYPES DE DONNEES

la matiere premiere de tout programme

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

42

int / entier

Nombre sans virgule.

02

3.14

float / decimal

Nombre a virgule.

03

"abc"

string / texte

Chaine de caracteres.

04

True

bool / booleen

Vrai ou Faux, rien d'autre.

05

[1,2]

list / tableau

Collection ordonnee, modifiable.

06

{k:v}

dict / objet

Paires cle-valeur.

07

(1,2)

tuple

Collection figee, non modifiable.

08

{1,2}

set / ensemble

Valeurs uniques, sans ordre.

09

None

null / vide

Absence volontaire de valeur.

10

const

constante

Valeur qui ne changera jamais.

// ASTUCE AREEJ CODE

En Python, tape type(x) pour decouvrir le type de n'importe quelle valeur. En JS : typeof x. Teste chaque type de cette page dans une console - 5 minutes qui valent un cours entier.

{ </> }

OPERATEURS D'AFFECTATION

ecrire moins, calculer plus

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

=

AFFECTER

x = 5 : x vaut 5.

02

+=

AJOUTER

x += 2 : x = x + 2.

03

-=

RETIRER

x -= 2 : x = x - 2.

04

*=

MULTIPLIER

x *= 3 : x = x * 3.

05

/=

DIVISER

x /= 2 : x = x / 2.

06

//

DIV. ENTIERE

7 // 2 = 3 (sans virgule).

07

**

PUISSANCE

2 ** 3 = 8.

08

%=

MODULO-AFFECTER

x %= 3 : reste de x / 3.

09

x++

INCREMENTER

Ajoute 1 (JS, C, Java..).

10

x - -

DECREMENTER

Retire 1 (JS, C, Java..).

// ASTUCE AREEJ CODE

x += 1 se lit : prends x, ajoute 1, remets le resultat dans x. Tous les operateurs composes suivent cette logique. Attention : = affecte, == compare. Les confondre est le bug n.1 des debutants.

{ </> }

LES CONDITIONS

le code qui decide

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

PYTHON

```python
age = 18
if age >= 18:
    print("majeur")
elif age >= 12:
    print("ado")
else:
    print("enfant")
```

JAVASCRIPT

```javascript
let age = 18;
if (age >= 18) {
  console.log("majeur");
} else if (age >= 12) {
  console.log("ado");
} else {
  console.log("enfant");
```

TERNAIRE - le if en 1 ligne

```
statut = 'majeur' if age >= 18 else 'mineur'   # Python
let statut = age >= 18 ? 'majeur' : 'mineur';  // JS
```

01

and / &&

LES DEUX

vrai si les 2 sont vraies

02

or / ||

AU MOINS UNE

vrai si une est vraie

03

not / !

INVERSE

vrai devient faux

04

match

SWITCH

1 cas parmi plusieurs

REGLE D'OR : chaque if doit pouvoir se lire a voix haute. Si tu ne peux pas l'expliquer, simplifie-le.

{ </> }

LES BOUCLES

repeter sans se repeter

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

FOR - quand on connait le nombre

```
for i in range(5):
    print(i)
# 0 1 2 3 4

for fruit in liste:
    print(fruit)
```

WHILE - tant que c'est vrai

```
n = 3
while n > 0:
    print(n)
    n -= 1
# 3 2 1
# attention boucle infinie!
```

01

break

STOP

sort de la boucle tout de suite

02

continue

SAUTER

passe au tour suivant

03

range(a,b)

COMPTEUR

de a jusqu'a b-1

04

enumerate

INDEX+VALEUR

i, v = position et valeur

LE PIEGE CLASSIQUE

```
# les index commencent a 0, pas a 1 !
liste = ['a','b','c']   # liste[0]='a'  liste[2]='c'
# liste[3] -> ERREUR : index out of range
```

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

LES FONCTIONS

ecrire une fois, utiliser partout

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

PYTHON

```python
def saluer(nom):
    return "Salam " + nom

msg = saluer("Areej")
# Salam Areej
```

JAVASCRIPT

```javascript
function saluer(nom) {
  return "Salam " + nom;
}

const f = (nom) => 'Salam '+nom;
```

01

def / function

DEFINIR

creer la recette

02

return

RENVOYER

la fonction rend un resultat

03

(a, b=2)

PARAMETRES

b=2 : valeur par defaut

04

lambda

MINI-FONCTION

fonction anonyme en 1 ligne

BONNE PRATIQUE

```python
# une fonction = UNE mission, un nom qui dit tout
def calculer_prix_ttc(prix_ht, tva=0.2):
    return prix_ht * (1 + tva)
```

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

LISTES & TABLEAUX

la structure que tu utiliseras chaque jour

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

`liste[0]`

PREMIER

Index 0 = premier element.

02

`liste[-1]`

DERNIER

Index -1 = dernier element.

03

`liste[1:4]`

TRANCHE

Elements 1, 2 et 3.

04

`append`

AJOUTER

Ajoute a la fin (push en JS).

05

`pop`

RETIRER

Enleve et renvoie un element.

06

`len(liste)`

TAILLE

Nombre d'elements.

07

`sort`

TRIER

Trie la liste sur place.

08

`x in liste`

CONTIENT ?

Vrai si x est dedans.

// ASTUCE AREEJ CODE

liste[1:4] prend les index 1, 2, 3 - la borne de droite est toujours EXCLUE. Et liste[::-1] renverse toute la liste d'un coup. Les slices sont la superpuissance cachee de Python.

{ </> }

LES CHAINES DE CARACTERES

maitrise le texte, maitrise les donnees

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

`a + b`

CONCATENER

Colle deux textes.

02

`f"{x}"`

F-STRING

Variable dans le texte (Python).

03

`` `${x}` ``

TEMPLATE JS

Variable dans le texte (JS).

04

`upper()`

MAJUSCULES

'ab' devient 'AB'.

05

`strip()`

NETTOYER

Retire les espaces autour (trim).

06

`split(',')`

DECOUPER

Texte vers liste.

07

`join(liste)`

RECOLLER

Liste vers texte.

08

`replace(a,b)`

REMPLACER

Change a en b partout.

// ASTUCE AREEJ CODE

Pipeline classique de nettoyage de donnees : texte.strip().lower().split(',') - nettoyer, uniformiser, decouper. 80% du travail sur du texte tient dans les 8 tuiles ci-dessus.

{ </> }

PYTHON EN 1 PAGE

un vrai mini-programme commente

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

depenses.py

```python
# gestionnaire de depenses - AREEJ CODE
depenses = []                        # liste vide

def ajouter(nom, prix):
    depenses.append({'nom': nom, 'prix': prix})

def total():
    return sum(d['prix'] for d in depenses)

ajouter('the', 12)
ajouter('msemen', 8)

for d in depenses:                   # boucle
    print(f"{d['nom']} : {d['prix']} DH")

if total() > 15:                     # condition
    print("Budget depasse !")
else:
    print("Tout va bien.")

# the : 12 DH / msemen : 8 DH / Budget depasse !
```

Recopie-le a la main, puis modifie-le : ajoute une depense, change le budget. C'est comme ca qu'on apprend.

{ </> }

JAVASCRIPT EN 1 PAGE

le meme programme, cote web

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

depenses.js

```
// gestionnaire de depenses - AREEJ CODE
const depenses = [];

function ajouter(nom, prix) {
  depenses.push({ nom, prix });
}

const total = () =>
  depenses.reduce((s, d) => s + d.prix, 0);

ajouter('the', 12);
ajouter('msemen', 8);

for (const d of depenses) {        // boucle
  console.log(`${d.nom} : ${d.prix} DH`);
}

if (total() > 15) {                // condition
  console.log('Budget depasse !');
} else {
  console.log('Tout va bien.');
}
```

Compare ligne par ligne avec la page 09 : memes idees, syntaxe differente. Les concepts sont universels.

{ </> }

HTML : 15 BALISES ESSENTIELLES

le squelette de toutes les pages web

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

`<html>`

LA PAGE

Racine de toute page web.

02

`<head>`

LA TETE

Infos invisibles : titre, styles.

03

`<body>`

LE CORPS

Tout ce qui s'affiche.

04

`<h1>`

GRAND TITRE

h1 a h6, du plus au moins grand.

05

`<p>`

PARAGRAPHE

Un bloc de texte.

06

`<a href>`

LIEN

Vers une autre page.

07

`<img src>`

IMAGE

Affiche une image.

08

`<div>`

BOITE

Conteneur pour organiser.

09

`<ul><li>`

LISTE

Liste a puces.

10

`<table>`

TABLEAU

Lignes et colonnes.

11

`<form>`

FORMULAIRE

Recueille des donnees.

12

`<input>`

CHAMP

Zone de saisie.

13

`<button>`

BOUTON

Declenche une action.

14

`<header>`

EN-TETE

Haut de page semantique.

15

`<footer>`

PIED

Bas de page semantique.

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

CSS : 15 PROPRIETES ESSENTIELLES

l'habillage de ta page

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

color

COULEUR TEXTE

color: #E5247E;

02

background

FOND

background: #0A0B10;

03

font-size

TAILLE TEXTE

font-size: 18px;

04

margin

MARGE EXTERNE

Espace autour de la boite.

05

padding

MARGE INTERNE

Espace dedans la boite.

06

border

BORDURE

border: 2px solid lime;

07

radius

COINS RONDS

border-radius: 12px;

08

flex

FLEXBOX

display:flex - aligner facile.

09

justify

AXE PRINCIPAL

justify-content: center;

10

align

AXE CROISE

align-items: center;

11

grid

GRILLE

display:grid - en colonnes.

12

position

POSITION

relative, absolute, fixed.

13

width

LARGEUR

width: 100%; max-width...

14

shadow

OMBRE

box-shadow: 0 4px 12px...

15

:hover

SURVOL

Style au passage de la souris.

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

GIT : 12 COMMANDES VITALES

git + commande, et ton travail est en securite

. SAUVEGARDE. VERSIONNE. COLLABORE. .

01

init

DEMARRER

Cree le depot dans le dossier.

02

clone

COPIER

Recupere un depot distant.

03

status

OU J'EN SUIS ?

Fichiers modifies, en attente.

04

add .

PREPARER

Met les fichiers dans le panier.

05

commit -m

ENREGISTRER

Sauvegarde avec un message.

06

push

ENVOYER

Monte tes commits en ligne.

07

pull

RECUPERER

Descend les nouveautes.

08

branch

BRANCHE

Liste ou cree des branches.

09

checkout

CHANGER

Bascule de branche.

10

merge

FUSIONNER

Reunit deux branches.

11

log

HISTORIQUE

Tous les commits.

12

.gitignore

IGNORER

Fichiers a ne jamais suivre.

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

LE TERMINAL SANS PEUR

12 commandes pour parler a ta machine

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

pwd

OU SUIS-JE ?

Affiche le dossier actuel.

02

ls

LISTER

Contenu du dossier.

03

cd

SE DEPLACER

cd dossier / cd .. remonte.

04

mkdir

CREER DOSSIER

mkdir mon-projet

05

touch

CREER FICHIER

touch index.html

06

cp

COPIER

cp source destination

07

mv

DEPLACER

Deplace ou renomme.

08

rm

SUPPRIMER

ATTENTION : definitif !

09

cat

AFFICHER

Montre le contenu d'un fichier.

10

clear

NETTOYER

Vide l'ecran.

11

history

HISTORIQUE

Tes dernieres commandes.

12

sudo

SUPER-POUVOIR

Execute en administrateur.

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

SQL : PARLER AUX DONNEES

10 mots-cles pour interroger une base

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

SELECT

CHOISIR

SELECT nom FROM clients;

02

WHERE

FILTRER

... WHERE ville = 'Casa';

03

ORDER BY

TRIER

... ORDER BY prix DESC;

04

LIMIT

LIMITER

... LIMIT 10;

05

INSERT

AJOUTER

INSERT INTO clients VALUES...

06

UPDATE

MODIFIER

UPDATE clients SET ...

07

DELETE

SUPPRIMER

Toujours avec un WHERE !

08

JOIN

RELIER

Combine deux tables.

09

GROUP BY

REGROUPER

Avec COUNT, SUM, AVG...

10

COUNT(*)

COMPTER

Nombre de lignes.

// ASTUCE AREEJ CODE

Ordre d'ecriture : SELECT ... FROM ... WHERE ... GROUP BY ... ORDER BY ... LIMIT. Et la regle de survie : jamais de DELETE ou UPDATE sans WHERE - sinon toute la table y passe.

{ </> }

REGEX : CHERCHER COMME UN PRO

le langage secret de la recherche de texte

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

.

N'IMPORTE QUEL

Un caractere, peu importe lequel.

02

^

DEBUT

^Bonjour : commence par.

03

$

FIN

maroc$: finit par.

04

*

0 OU PLUS

Repete 0 fois ou plus.

05

+

1 OU PLUS

Repete au moins 1 fois.

06

?

OPTIONNEL

0 ou 1 fois.

07

[abc]

UN PARMI

a, b ou c.

08

[0-9]

UN CHIFFRE

N'importe quel chiffre.

09

\d

CHIFFRE

Raccourci de [0-9].

10

\w

LETTRE/CHIFFRE

Caractere de mot.

11

\s

ESPACE

Espace, tab, retour ligne.

12

{2,4}

ENTRE 2 ET 4

Nombre de repetitions.

13

a|b

OU

a ou bien b.

14

( )

GROUPE

Capture une partie.

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

HTTP : LE WEB TE REPOND

codes et verbes des API

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

200

OK

Tout s'est bien passe.

02

201

CREE

Ressource creee avec succes.

03

301

DEPLACE

Redirection permanente.

04

400

MAUVAISE REQUETE

Le client a mal demande.

05

401

NON CONNECTE

Il faut s'identifier.

06

403

INTERDIT

Identifie mais pas autorise.

07

404

INTROUVABLE

La page n'existe pas.

08

500

ERREUR SERVEUR

Le probleme vient du serveur.

09

GET

LIRE

Recuperer des donnees.

10

POST

CREER

Envoyer des donnees.

11

PUT

REMPLACER

Mettre a jour en entier.

12

DELETE

SUPPRIMER

Effacer la ressource.

// ASTUCE AREEJ CODE

Memo : 2xx = succes, 3xx = redirection, 4xx = erreur du client (toi), 5xx = erreur du serveur (eux). Un GET ne doit JAMAIS modifier des donnees - c'est le contrat du web.

{ </> }

RACCOURCIS VS CODE

code 2 fois plus vite des aujourd'hui

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

01

Ctrl+S

SAUVEGARDER

Le reflexe n.1 du developpeur.

02

Ctrl+C/V

COPIER-COLLER

Le duo legendaire.

03

Ctrl+Z

ANNULER

Ctrl+Shift+Z pour refaire.

04

Ctrl+/

COMMENTER

Commente la ligne selectionnee.

05

Ctrl+D

SELECTION MULTI

Selectionne le mot suivant identique.

06

Alt+Fleches

DEPLACER LIGNE

Monte ou descend la ligne.

07

Ctrl+P

OUVRIR FICHIER

Recherche rapide de fichier.

08

Ctrl+Shift+P

PALETTE

Toutes les commandes VS Code.

09

Ctrl+F

CHERCHER

Ctrl+H pour remplacer.

10

F12

ALLER A LA DEF

Saute vers la definition.

11

Ctrl+`

TERMINAL

Ouvre le terminal integre.

12

F5

DEBUGGER

Lance le programme pas a pas.

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.

{ </> }

QUIZ + PENSE-BETE

page de travail - a toi de jouer

. MAITRISE LES BASES. CONSTRUIS LE FUTUR. .

1. RELIE chaque symbole a son role (au crayon) :

`&&`	○	○	Fin d'instruction
`%`	○	○	Different de
`[ ]`	○	○	Et logique
`=>`	○	○	Tableaux et index
`!=`	○	○	Reste d'une division
`;`	○	○	Fonction flechee JS

2. TES 8 TUILES A TOI – note ce que TU oublies toujours :

01

02

03

04

05

06

07

08

CONTINUE D'APPRENDRE. CONTINUE DE CONSTRUIRE. CONTINUE DE GRANDIR.